Student: Andrius Klimas

D number: D00246310

Lecturer: Andrew Shaw

Subject: Deployment

Date Due: 10/05/2026

Course: System Operations and Development

Introduction

In this document I will be going over how to upload a project to Kubernetes and azure. I will be also going over the changed needed to make this happen as well as any automation that was done to get the project fast tracked and to reduce repetitive inputs.

Automation

Pipeline

To do this I used 2 different files, they are called .gitlab-ci.yml which is created when setting up a pipeline on gitlab, and I will be using build.py.

build.py | docker-compose.yml | dockerfile song-player-svc | dockerfile shop-front-svc | dockerfile

build.py > build_and_push_images

```python

def build_and_push_images(tag):
    # Map folder names to image names
    images = {
        'catalog-svc': 'catalog-svc_1',
        'history-svc': 'history-svc',
        'login-page-svc': 'login-page-svc',
        'product-admin-svc': 'product-admin-svc',
        'shop-front-svc': 'shop-front-svc',
        'song-player-svc': 'song-player-svc',
        'gateway': 'gateway'
    }
    docker_registry = 'andriuskl'

    for folder, image_repo in images.items():
        image_name = f"{docker_registry}/{image_repo}:{tag}"
        print(f"Building {image_name}...")

        # Build the Docker image
        build_command = f"docker build -t {image_name} ./{folder}"
        result = subprocess.run(build_command, shell=True)
        if result.returncode != 0:
            print(f"Warning: Build failed for {image_repo}, skipping push...")
            continue

        # Push the Docker image to Dockerhub
        print(f"Pushing {image_name}...")
        push_command = f"docker push {image_name}"
        result = subprocess.run(push_command, shell=True)
        if result.returncode != 0:
            print(f"Error: Push failed for {image_repo}")
        else:
            print(f"Successfully pushed {image_name}")


```

```python
if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Usage: python build.py <tag>")
        sys.exit(1)

    tag = sys.argv[1]


    build_and_push_images(tag,)
```

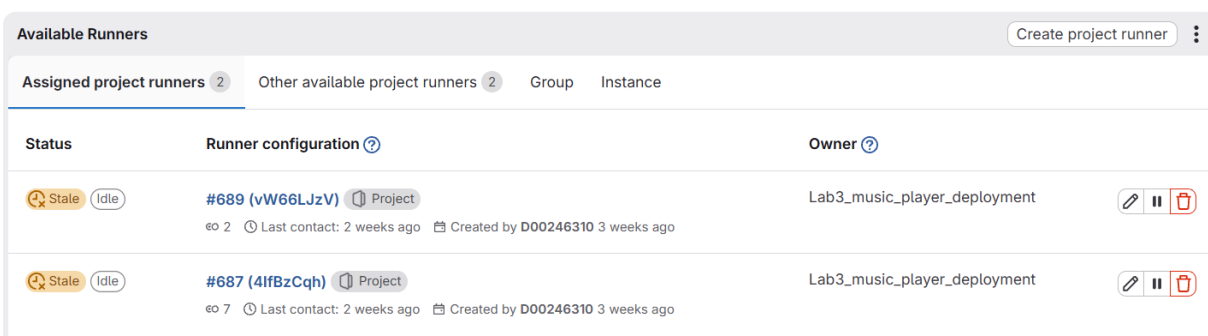
.gitlab-ci.yml > { } build-job > [ ] rules > { } 0

gitlab-ci - GitLab CI Configuration file (ci.json)

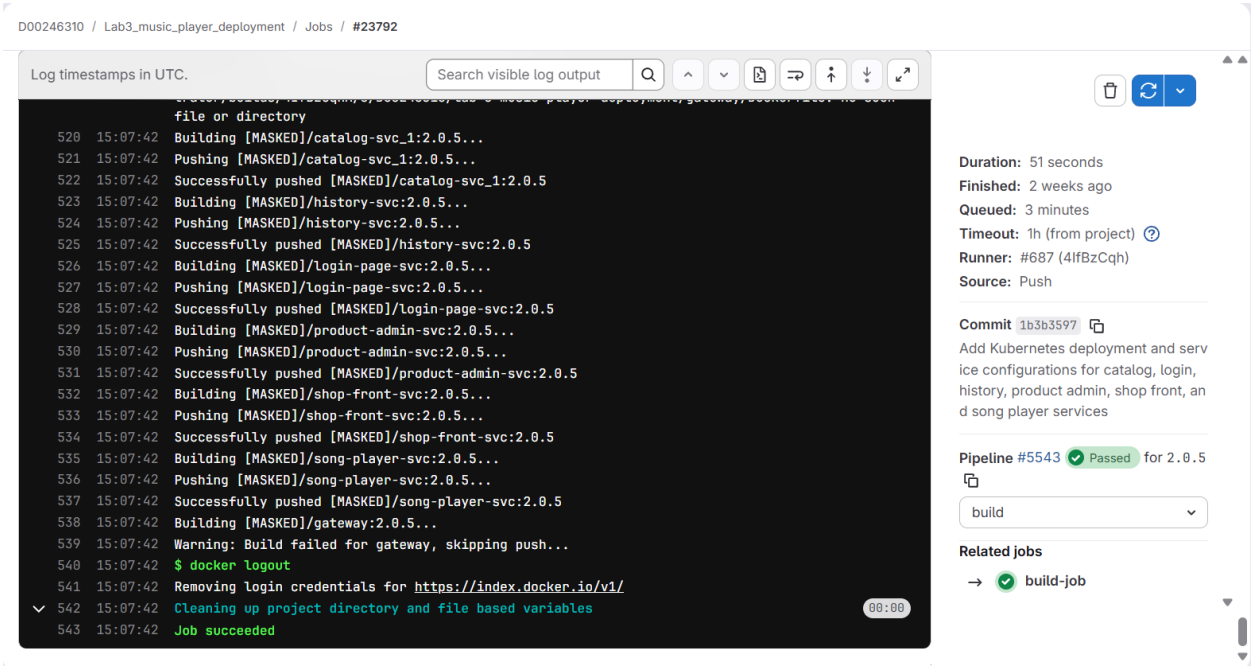
```yaml
stages:
  - build

build-job:
  stage: build
  script:
    - echo "Starting build job"
    - echo $SHELL
    - bash --version
    - python3 --version
    - echo "Environment variables set"
    - echo $DOCKER_PASSWORD | docker login -u $DOCKER_USERNAME --password-stdin
    - python3 build.py $CI_COMMIT_TAG
    - docker logout
  rules:
    - if: $CI_COMMIT_TAG
```

These 2 files work together to create an image of the files specified and upload them to docker hub under the account specified.

Below you can see the runners, they are currently offline as I do not have the server I am using online.



Below you can see the pipeline running, it is making a image of the specific docker containers. The tag is 2.0.5



Here you can see them on docker hub

Repositories

All repositories within the **andriuskl** namespace.

Search by repository name

All content

Name
andriuskl/**song-player-svc**
andriuskl/**shop-front-svc**
andriuskl/**product-admin-svc**
andriuskl/**login-page-svc**
andriuskl/**history-svc**
andriuskl/**catalog-svc_1**
andriuskl/**catalog-svc**
andriuskl/**simple-server**

Repositories / song-player-svc / General

andriuskl/song-player-svc

Last pushed 15 days ago • Repository size: 385 MB • 0 • 88

Add a description

Add a category

General | Tags | Image Management | Collaborators | Webhooks | Settings

Tags

DOCKER SCOUT INACTIVE
Activate

This repository contains 4 tag(s).

Tag	OS	Type	Pulled	Pushed
2.0.5		Image	less than 1 day	15 days

Manifests

I have 8 pairs of manifests.

Catalog-db service and deploy

Login-db service and deploy

Catalog-svc service and deploy

Shop-front-svc service and deploy

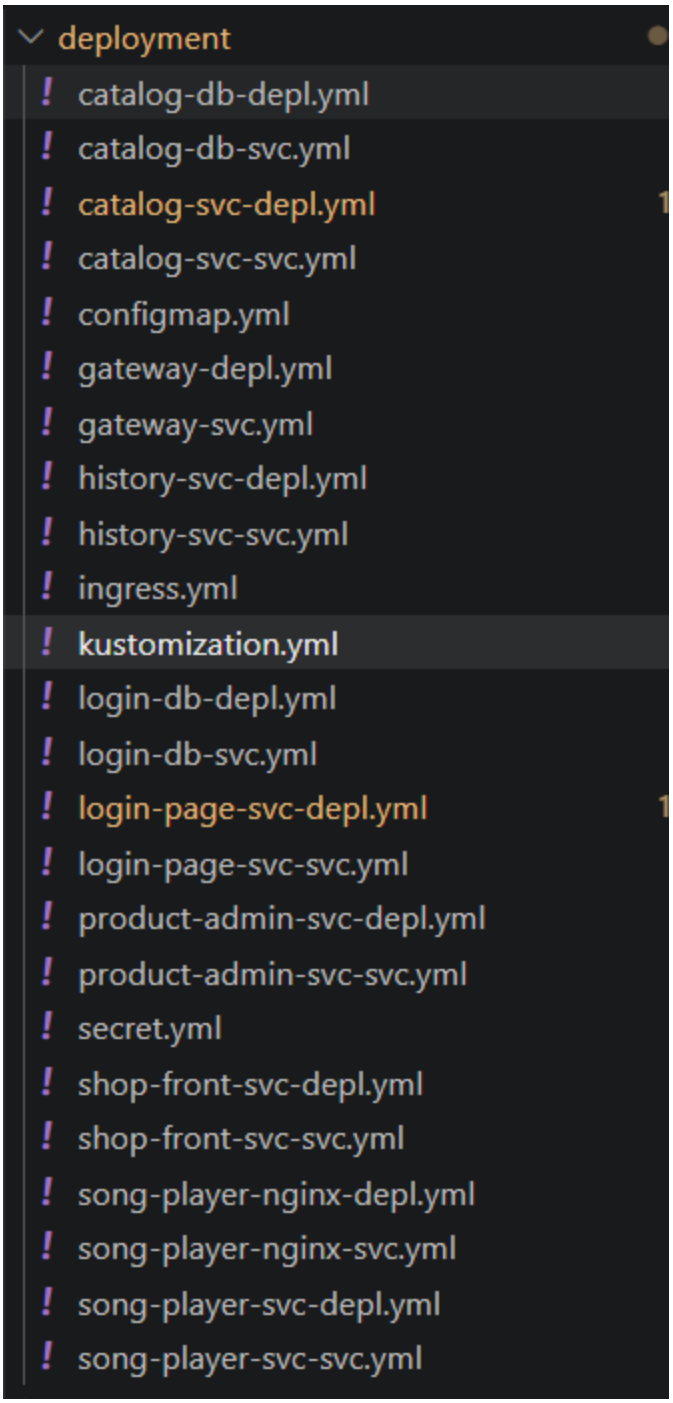
Product-admin-svc service and deploy

Song-player-svc service and deploy

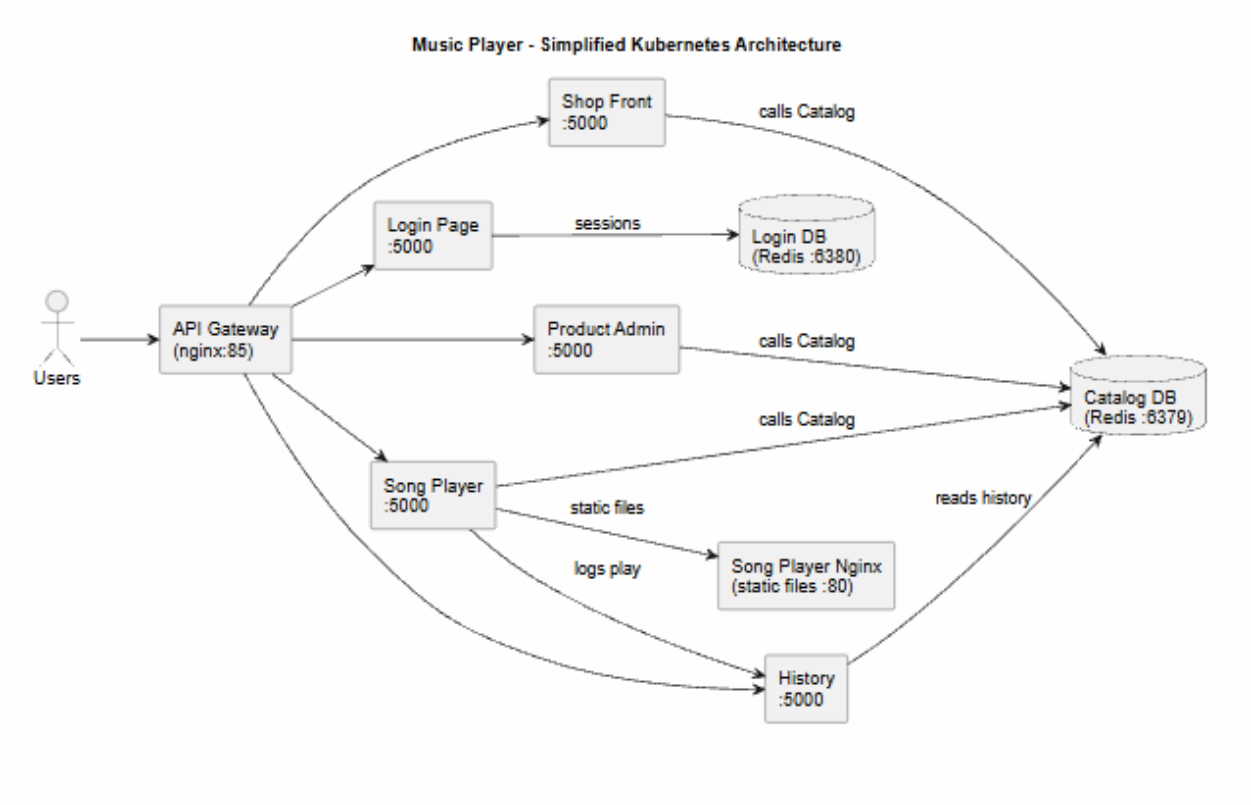
History-svc service and deploy

Login-page-svc service and deploy

Please see images below



And the image below explains how they all interact with each other



Kustomize setup

Deployment

In the kustomize file in the deployment it just has all the manifests needed to get the application to run, this includes all the 8 pairs aswell as configmap and secrets and gateway. These are used to to push these images to kubernets

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: music-app

resources:
  - configmap.yml
  - secret.yml
  - catalog-db-depl.yml
  - catalog-db-svc.yml
  - login-db-depl.yml
  - login-db-svc.yml
  - catalog-svc-depl.yml
  - catalog-svc-svc.yml
  - shop-front-svc-depl.yml
  - shop-front-svc-svc.yml
  - product-admin-svc-depl.yml
  - product-admin-svc-svc.yml
  - song-player-svc-depl.yml
  - song-player-svc-svc.yml
  - history-svc-depl.yml
  - history-svc-svc.yml
  - login-page-svc-depl.yml
  - login-page-svc-svc.yml
  - song-player-nginx-depl.yml
  - song-player-nginx-svc.yml
  - gateway-depl.yml
  - gateway-svc.yml
#  - ingress.yml
```

Dev

In the dev it has the version being used in the dev environment aswell as in the top it specifies where you can locate the actual manifests in the deployment folder and specifying the ingress file to use

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - ./../deployment
  - ./ingress-gateway.yml



images:
- name: andriuskl/shop-front-svc
  newName: andriuskl/shop-front-svc
  newTag: 2.0.5
- name: andriuskl/product-admin-svc
  newName: andriuskl/product-admin-svc
  newTag: 2.0.5
- name: andriuskl/catalog-svc
  newName: andriuskl/catalog-svc
  newTag: 2.0.5
# - name: andriuskl/gateway
#   newName: andriuskl/gateway
#   newTag: 2.0.5
- name: andriuskl/history-svc
  newName: andriuskl/history-svc
  newTag: 2.0.5
- name: andriuskl/login-page-svc
  newName: andriuskl/login-page-svc
  newTag: 2.0.5
- name: andriuskl/song-player-svc
  newName: andriuskl/song-player-svc
  newTag: 2.0.5
```

Prod

This is the same for the kuztomisation file in the prod folder

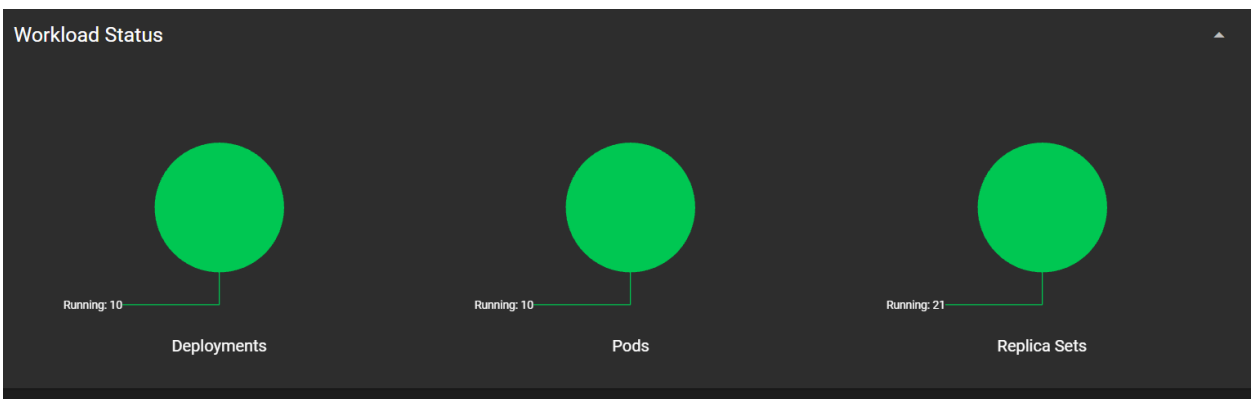
```yaml
kustomization.yaml - Kubernetes native configuration management (kustomization.json)
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - ./../deployment
  - ./azure-ingress-gateway.yml



images:
- name: andriuskl/shop-front-svc
  newName: andriuskl/shop-front-svc
  newTag: 2.0.5
- name: andriuskl/product-admin-svc
  newName: andriuskl/product-admin-svc
  newTag: 2.0.5
- name: andriuskl/catalog-svc
  newName: andriuskl/catalog-svc_1
  newTag: 2.0.5
# - name: andriuskl/gateway
#   newName: andriuskl/gateway
#   newTag: 2.0.5
- name: andriuskl/history-svc
  newName: andriuskl/history-svc
  newTag: 2.0.5
- name: andriuskl/login-page-svc
  newName: andriuskl/login-page-svc
  newTag: 2.0.5
- name: andriuskl/song-player-svc
  newName: andriuskl/song-player-svc
  newTag: 2.0.5
```

Screenshots of full deployment in Minkube Dashboard.

Below you can see all the services and deployments running correctly and online

```
🚜 Pulling base image v0.0.50 ...
🔄 Restarting existing docker container for "minikube" ...
🐳 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔎 Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
💡 After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
    ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
    ▪ Using image registry.k8s.io/ingress-nginx/controller:v1.14.3
    ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.7
    ▪ Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
    ▪ Using image docker.io/kubernetesui/dashboard:v2.7.0
🔎 Verifying ingress addon...
💡 Some dashboard features require the metrics-server addon. To enable all features please run:

        minikube addons enable metrics-server

🌟 Enabled addons: storage-provisioner, dashboard, ingress, default-storageclass

❗ C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.32.2, which may have incompatibilities with Kubernetes 1.35.1.
    ▪ Want kubectl v1.35.1? Try 'minikube kubectl -- get pods -A'
🏄 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Program Files\Kubernetes\Minikube>
```

```
PS C:\Users\andri\OneDrive\Desktop\YEAR2SEM2\Deployment\lab-3-music-player-deployment> minikube dashboard
🤔 Verifying dashboard health ...
🚀 Launching proxy ...
🤔 Verifying proxy health ...
🎉 Opening http://127.0.0.1:54496/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...
```



Deployments

Name	Images	Labels	Pods	Created
login-page-svc-depl	andriuskl/login-page-svc:2.0.5	-	1 / 1	14 days ago
song-player-nginx-depl	nginx:latest	-	1 / 1	14 days ago
song-player-svc-depl	andriuskl/song-player-svc:2.0.5	-	1 / 1	14 days ago
product-admin-svc-depl	andriuskl/product-admin-svc:2.0.5	-	1 / 1	14 days ago
shop-front-svc-depl	andriuskl/shop-front-svc:2.0.5	-	1 / 1	14 days ago
history-svc-depl	andriuskl/history-svc:2.0.5	-	1 / 1	14 days ago
catalog-svc-depl	andriuskl/catalog-svc_1:2.0.5	-	1 / 1	15 days ago
login-db-depl	redis:8.6	-	1 / 1	15 days ago
catalog-db-depl	redis:8.6	-	1 / 1	15 days ago
gateway-depl	nginx:latest	-	1 / 1	15 days ago

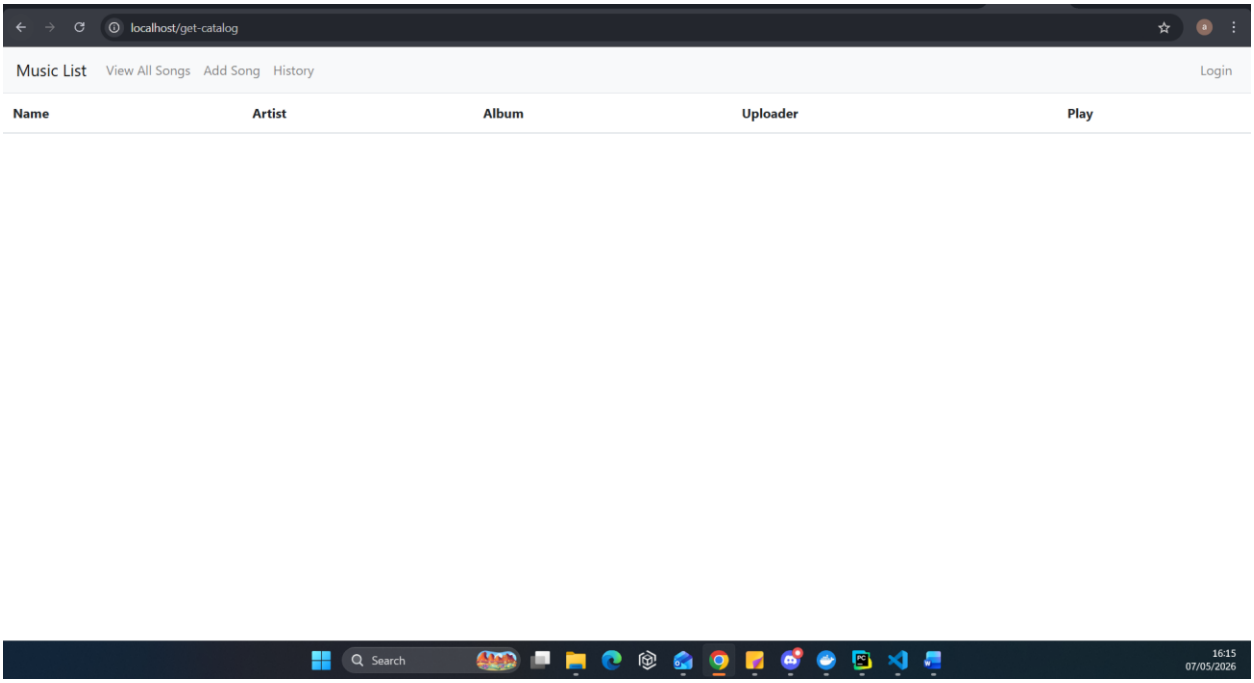
catalog-db	-	ClusterIP	10.109.8.63	catalog-db.music-app:6379 TCP catalog-db.music-app:0 TCP	-	15 days ago
catalog-svc	-	ClusterIP	10.109.171.3	catalog-svc.music-app:5000 TCP catalog-svc.music-app:0 TCP	-	15 days ago
gateway	-	ClusterIP	10.102.228.245	gateway.music-app:85 TCP gateway.music-app:0 TCP	-	15 days ago
history-svc	-	ClusterIP	10.105.17.3	history-svc.music-app:5000 TCP history-svc.music-app:0 TCP	-	15 days ago
login-db	-	ClusterIP	10.110.16.182	login-db.music-app:6379 TCP login-db.music-app:0 TCP	-	15 days ago
login-page-svc	-	ClusterIP	10.107.251.86	login-page-svc.music-app:5000 TCP login-page-svc.music-app:0 TCP	-	15 days ago
product-admin-svc	-	ClusterIP	10.107.164.23	product-admin-svc.music-app:5000 TCP product-admin-svc.music-app:0 TCP	-	15 days ago
shop-front-svc	-	ClusterIP	10.105.87.71	shop-front-svc.music-app:5000 TCP shop-front-svc.music-app:0 TCP	-	15 days ago
song-player-nginx	-	ClusterIP	10.99.98.225	song-player-nginx.music-app:8081 TCP song-player-nginx.music-app:0 TCP	-	15 days ago
song-player-svc	-	ClusterIP	10.97.230.23	song-player-svc.music-app:5000 TCP song-player-svc.music-app:0 TCP	-	15 days ago

Aswell you can see ingress running.

Ingresses

Name	Labels	Endpoints	Hosts	Created
app-ingress	-	192.168.49.2	localhost	8 days ago

And below you can see I am able to hop onto the website via the minikube tunnel



Description of non functional testing

Non functional testing is testing how well the application will run and not testing if each feature works. So I would do the following tests:

Load test: to check even with an influx of users will the applications response time stay at an acceptable level

Stress test: similar to load test, I would send a lot of requests at the same time and check if the system crashes due to an overload of requests (adding songs)

Performance test: to check the response time of key actions such as logging in, adding a song etc

Scalability: to check if the pods automatically scale up or down depending on the traffic as it is running on Kubernetes/azure

Reliability: to check if my system recovers effectively after a pod crashes (forced crash the pod if needed)

Security test: To check if there are any security issues or any information that is given to the user that should not be e.g. other users login information

The 2 applications that can be used for this purpose are

Apache JMeter as it is an open source application that can be used by anyone and it has built in reports and can easily be integrated into the system.

Locust as it uses they python language and since the application is made using python then the integration should be easy and it is open source.

Description of deployment strategies

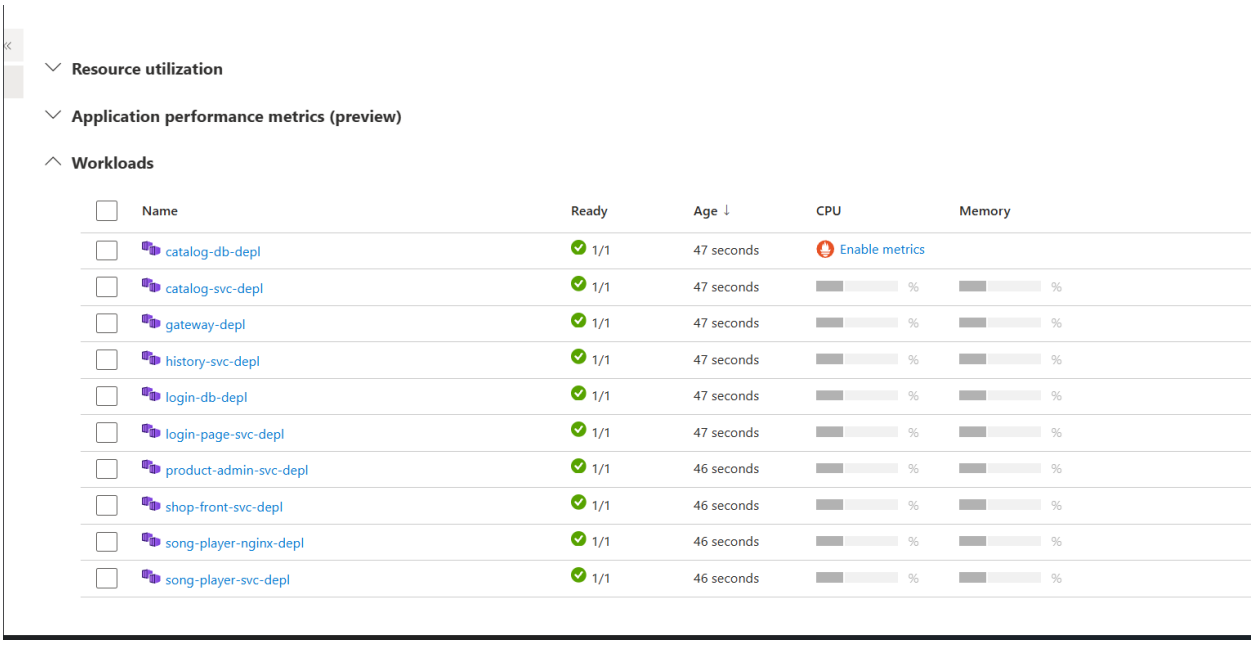
The music player uses automation in the pipeline process, where when a new version is ready to be uploaded onto docker hub, I can create a tag e.g. 2.0.6, it will create that and upload it to docker hub for me. By using Kubernetes I can control the version that is being used by changing it in the respective kuztomisation file depending on the environment e.g. dev or prod. And when I push to those environments the current pods will be replaced with the new ones with almost zero down time. This will only work with individual pods, for larger scale changes we may need to implement a blue-green strategy, where we have 2 identical environments and we want to make changes on blue, we route all the traffic to green, and then start on the changes to blue, and switch over back to blue when changes are done so green can also be updated. Aswell we can test all the changes in dev before pushing them to prod so If anything happens we can just move to the newer version. And since Kubernetes is pulling from docker hub, if the newest version e.g. 2.0.6 runs into issues we can put it back to 2.0.5 and have it on a more stable version until the initial issue is located and fixed and a newer e.g. 2.0.7 version is released so it can be tested and released to prod.

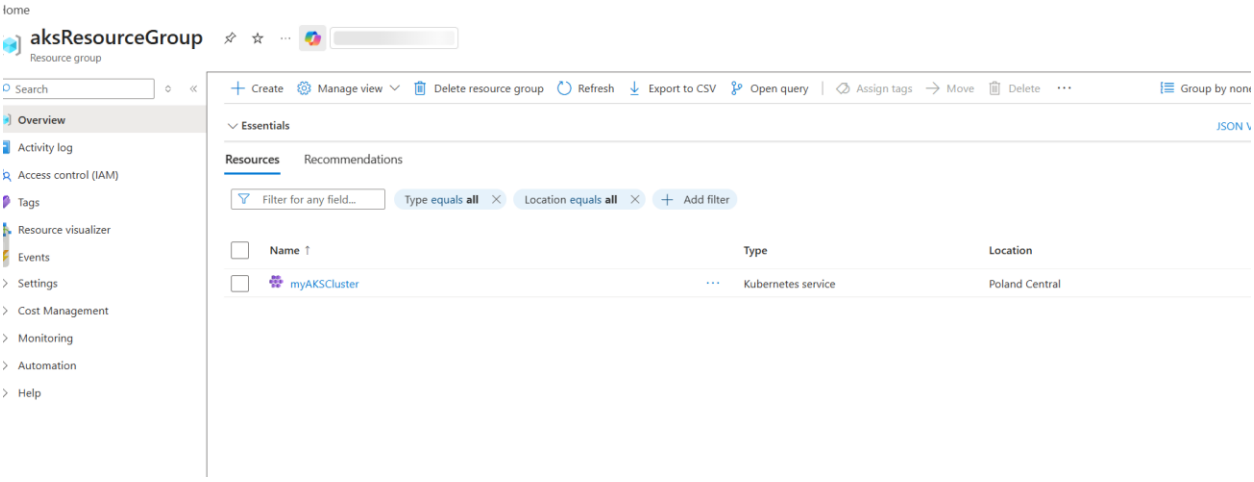
Conclusion

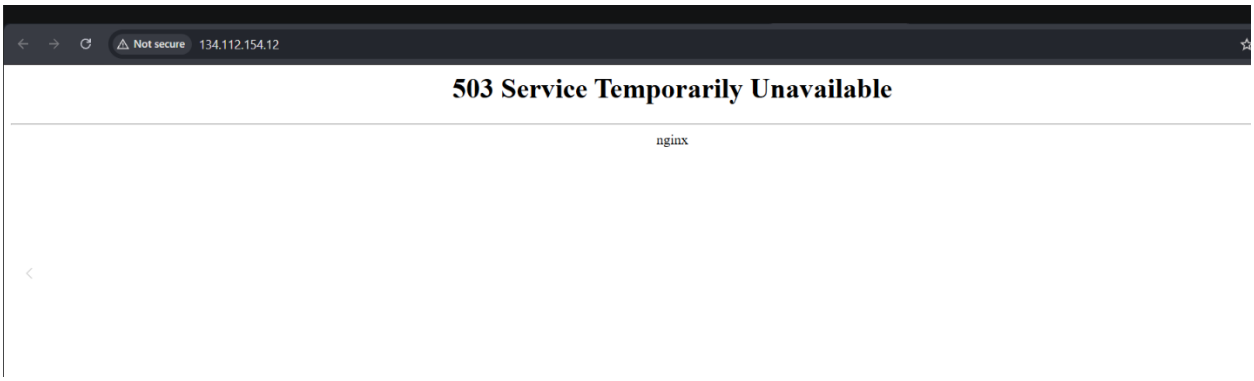
In conclusion, this shows the automation process implemented, a brief overview of all the manifest files and how they interact with each other, the kustomization files and the different environments that there are and how they are used for each one, the project running in Kubernetes (all pods green and services running and being able to connect to it via the web), as well as an brief explanation of the non functions testing that I would do and the deployment strategies. P.S. no code was changes as we didn’t add anything new into the main code.

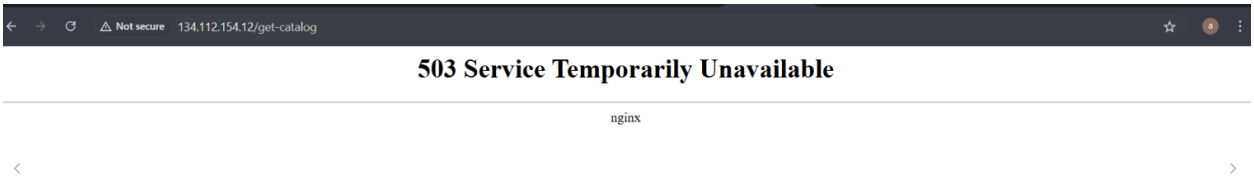
Any major issues encountered

I ran into 1 major issue when trying to get the application running onto azure, it was all added correctly, the resource group and the clusters. But when trying to connect via the ip it would give me a 503 error. I tried uploading a new ingress file but it was still the same.









Even after rerunning ingress issue persisted. I could not find the root cause to this. It is an issue with ingress but it is working correctly in Kubernetes. The only change I made was change the below

```
io.k8s.api.networking.v1.Ingress (v1@ingress.json)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "8m"
spec:
  rules:
  - host: localhost
    http:
      paths:
      - pathType: Prefix
        path: /get-catalog
        backend:
```

To

```yaml
metadata:
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "8m"
    kubernetes.azure.com/app-routing-zone: ""
spec:
  ingressClassName: webapprouting.kubernetes.azure.com
  rules:
  - http:
      paths:
      - pathType: Prefix
        path: "/get-catalog"
        backend:
          service:
            name: shop-front-svc
```

And it would not work, as Kubernetes is using the first one and not the second one.

URL for GitLab

https://gitlab.comp.dkit.ie/D00246310/lab-3-music-player-deployment.git